

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 9

1 - Enunciado

Escreva um programa que cria tarefas para realizar N inserções em duas listas encadeadas, cada uma associada a uma thread. Cada tarefa deve escolher aleatoriamente em qual lista inserir um número. Garanta a integridade das listas evitando condição de corrida e, sempre que possível, use regiões críticas nomeadas para que a inserção em uma lista não bloqueie a outra. Em seguida, generalize o programa para um número de listas definido pelo usuário. Explique por que, neste caso, regiões críticas nomeadas não são suficientes e por que o uso de locks explícitos se torna necessário.

2 - Metodologia

No enunciado da tarefa fala sobre a implementação de duas listas encadeadas essas que devem receber N inserções, as inserções poderia ser feita de duas formas ou por uma variável definida em código ou pela entrada do usuário nesse caso foi usado o valor padrão N = 20, caso não seja inserido nenhuma quantidade de itens para a listas, para a compilação foram usadas as seguintes linhas de código:

```
gcc -fopenmp main_two_list.c -o main_2L  
export OMP_NUM_THREADS=12; time ./main_2L 4
```

Como os exemplos anteriores foi configurado a quantidade de 12 threads para a execução do código.

Na segunda implementação que era necessário a entrada do usuário com a quantidade de listas a serem criadas foi usada as seguintes linhas para a sua execução:

```
gcc -fopenmp main_n_lists.c -o main_NL
export OMP_NUM_THREADS=12; time ./main_NL 4 4
```

2.1- Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

3 - Código Fonte

3.1 - Código(task com 2 lista)

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <time.h>
#include <omp.h>

typedef struct No {
    int num;
    struct No *prox;
}No;

No *init(const int *num){
    No *novoNo = (No *) malloc(sizeof(No));
    if(novoNo == NULL){
        printf("Erro!");
        return NULL;
    }

    novoNo->num = *num;
    novoNo -> prox = NULL;
    return novoNo;
}

void add(No **cabeca, const int *num){
    No *novoNo = init(num);
    if (*cabeca == NULL){
        *cabeca = novoNo;
        return;
    }

    No *noAtual = *cabeca;

    while(noAtual -> prox !=NULL){

```

```

        noAtual = noAtual->prox;
    }
    noAtual->prox = novoNo;
}

int verificar(const No *cabeca){
    int count = 0;
    const No *atual = cabeca;
    while (atual != NULL){
        count++;
        atual = atual->prox;
    }
    return count;
}

void liberar(No *cabeca){
    No *noAtual = cabeca;

    while(noAtual != NULL){
        No *proximo = noAtual->prox;
        free(noAtual);
        noAtual=proximo;
    }
}

void processar(const No *no, int lista_escolhida){
    printf("Numero: %5d | lista: %d | thread: %d\n", no->num, lista_escolhida,
omp_get_thread_num());
}

```

```
int main(int argc, char *argv[]){
    int N = (argc > 1) ? atoi(argv[1]): 20;
    No *lista1 = NULL;
    No *lista2 = NULL;

    printf("Quantidade de numeros: %d \n", N);

    #pragma omp parallel
    {
        #pragma omp single
        {
            for (int i = 0; i < N; i++){
                #pragma omp task firstprivate(i)
                {
                    unsigned int seed = (unsigned int) time(NULL) ^ (unsigned int
)(omp_get_thread_num() << 12) ^ (unsigned int) i;

                    int valor = i + 1;

                    int lista_escolhida = rand_r(&seed) % 2;

                    No temp = {valor, NULL};

                    processar(&temp, lista_escolhida);

                    if (lista_escolhida == 0){
                        #pragma omp critical(lock_lista1)
                        {
                            add(&lista1, &valor);
                        }
                    }else{
                        #pragma omp critical(lock_lista2)
                        {
```

```

        add(&lista2, &valor);
    }
}

#pragma omp taskwait
}

}

int total1 = verificar(lista1);
int total2 = verificar(lista2);

printf("Lista 1: %d elementos\n", total1);
printf("Lista 2: %d elementos\n", total2);
printf("Total inserido: %d (esperado: %d)\n", total1+ total2, N);

liberar(lista1);
liberar(lista2);

return 0;
}

```

3.2 - código task com N entradas para listas

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <omp.h>

```

```
typedef struct No {  
    int num;  
    struct No *prox;  
}No;  
  
No *init(const int *num){  
    No *novoNo = (No *) malloc(sizeof(No));  
    if(novoNo == NULL){  
        printf("Erro!");  
        return NULL;  
    }  
  
    novoNo->num = *num;  
    novoNo -> prox = NULL;  
    return novoNo;  
}  
  
void add(No **cabeca, const int *num){  
    No *novoNo = init(num);  
    if (*cabeca == NULL){  
        *cabeca = novoNo;  
        return;  
    }  
  
    No *noAtual = *cabeca;  
  
    while(noAtual -> prox !=NULL){  
        noAtual = noAtual->prox;  
    }  
    noAtual->prox = novoNo;
```

```
}
```

```
int verificar(const No *cabeca){
```

```
    int count = 0;
```

```
    const No *atual = cabeca;
```

```
    while (atual != NULL){
```

```
        count++;
```

```
        atual = atual->prox;
```

```
    }
```

```
    return count;
```

```
}
```

```
void liberar(No *cabeca){
```

```
    No *noAtual = cabeca;
```

```
    while(noAtual != NULL){
```

```
        No *proximo = noAtual->prox;
```

```
        free(noAtual);
```

```
        noAtual=proximo;
```

```
    }
```

```
}
```

```
void processar(const No *no, int lista_escolhida){
```

```
    printf("Numero: %5d | lista: %d | thread: %d\n", no->num, lista_escolhida,
```

```
omp_get_thread_num());
```

```
}
```

```
int main(int argc, char *argv[]){
```

```
    int N = (argc > 1) ? atoi(argv[1]): 20;
```

```

int M = (argc > 2) ? atoi(argv[2]): 2;

printf("Quantidade de numeros: %d \n | Numero de Listas: %d", N, M);

No **listas = (No **) calloc(M, sizeof(No *));
omp_lock_t *locks = (omp_lock_t *) malloc(M * sizeof(omp_lock_t));

for(int j = 0; j < M; j++){
    omp_init_lock(&locks[j]);
}

#pragma omp parallel
{
    #pragma omp single
    {
        for (int i = 0; i < N; i++){
            #pragma omp task firstprivate(i)
            {
                unsigned int seed = (unsigned int) time(NULL) ^ (unsigned int
)(omp_get_thread_num() << 12) ^ (unsigned int) i;

                int valor = i + 1;

                int lista_escolhida = rand_r(&seed) % M;

                No temp = {valor, NULL};

                processar(&temp, lista_escolhida);

                omp_set_lock(&locks[lista_escolhida]);
                add(&listas[lista_escolhida], &valor);
                omp_unset_lock(&locks[lista_escolhida]);
            }
        }
    }
}

```

```

    }

}

}

#pragma omp taskwait

}

int total = 0;
for (int k = 0 ; k < M; k++){
    int n_listas = verificar(listas[k]);
    printf("Lista %d: %d elementos \n", k, n_listas);
    total += n_listas;
}

printf("Total inserido: %d (esperado: %d)\n", total, N);

for (int k = 0 ; k < M; k++){
    liberar(listas[k]);
    omp_destroy_lock(&locks[k]);
}

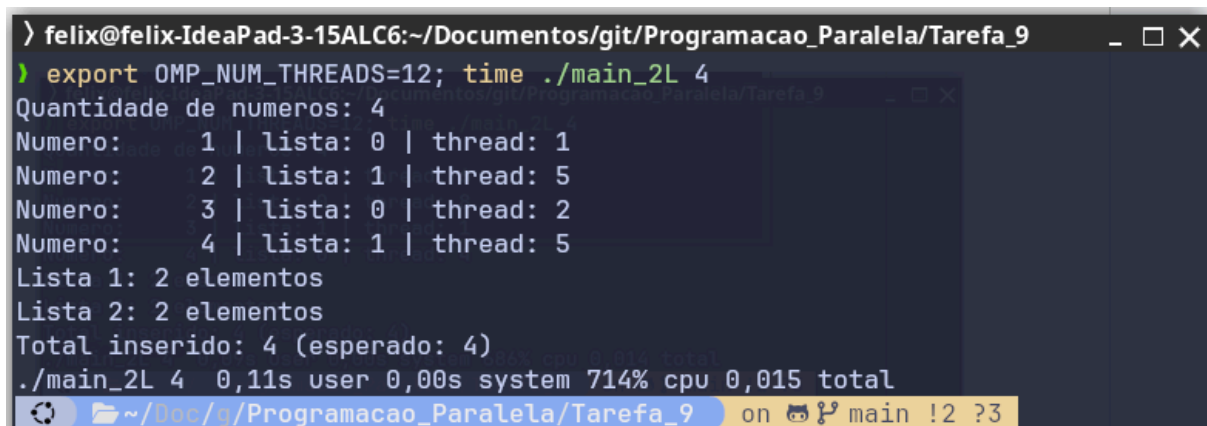
free(listas);
free(locks);

return 0;
}

```

4 - Resultado

Como o enunciado descreve duas implementações para os teste, a primeira usando `#pragma omp critical <nomeado>` teve a seguinte saída na figura 1 para uma entrada de $N = 4$.



```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_9
) export OMP_NUM_THREADS=12; time ./main_2L 4
Quantidade de numeros: 4
Numero: 1 | lista: 0 | thread: 1
Numero: 2 | lista: 1 | thread: 5
Numero: 3 | lista: 0 | thread: 2
Numero: 4 | lista: 1 | thread: 5
Lista 1: 2 elementos
Lista 2: 2 elementos
Total inserido: 4 (esperado: 4)
./main_2L 4 0,11s user 0,00s system 714% cpu 0,015 total
```

Figura 1: main_two_list.c com $N = 4$

Feito esses testes foi utilizado um número maior de para inserções na lista de maneira a verificar se o comportamento permaneceria como o estado anterior ou se aumentando a quantidade de inserções na lista isso poderia causar algum tipo de inconsistência.

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_9
> export OMP_NUM_THREADS=12; time ./main_2L 20
Quantidade de numeros: 20
Numero: 1 | lista: 0 | thread: 9
Numero: 2 | lista: 1 | thread: 2
Numero: 3 | lista: 0 | thread: 4
Numero: 4 | lista: 0 | thread: 2
Numero: 5 | lista: 0 | thread: 4
Numero: 6 | lista: 0 | thread: 4
Numero: 7 | lista: 1 | thread: 10
Numero: 8 | lista: 1 | thread: 4
Numero: 9 | lista: 0 | thread: 9
Numero: 10 | lista: 0 | thread: 9
Numero: 11 | lista: 0 | thread: 2
Numero: 12 | lista: 0 | thread: 2
Numero: 13 | lista: 0 | thread: 2
Numero: 14 | lista: 1 | thread: 10
Numero: 15 | lista: 1 | thread: 9
Numero: 16 | lista: 0 | thread: 9
Numero: 17 | lista: 1 | thread: 9
Numero: 18 | lista: 0 | thread: 10
Numero: 19 | lista: 0 | thread: 9
Numero: 20 | lista: 0 | thread: 10
Lista 1: 14 elementos
Lista 2: 6 elementos
Total inserido: 20 (esperado: 20)
./main_2L 20 0,14s user 0,00s system 815% cpu 0,018 total
```

Figura 2: main_two_list.c com N = 20

Para testar a implementação com um N = 100000 para o caso em que temos uma quantidade grande de inserções nessas listas:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_9 _ □ X
Numero: 99963 | lista: 0 | thread: 2
Numero: 99964 | lista: 0 | thread: 2
Numero: 99965 | lista: 1 | thread: 5
Numero: 99966 | lista: 1 | thread: 5
Numero: 99967 | lista: 0 | thread: 5
Numero: 99968 | lista: 0 | thread: 11
Numero: 99969 | lista: 1 | thread: 5
Numero: 99970 | lista: 1 | thread: 2
Numero: 99971 | lista: 0 | thread: 5
Numero: 99972 | lista: 0 | thread: 5
Numero: 99973 | lista: 0 | thread: 2
Numero: 99974 | lista: 1 | thread: 4
Numero: 99975 | lista: 0 | thread: 4
Numero: 99976 | lista: 0 | thread: 4
Numero: 99977 | lista: 0 | thread: 4
Numero: 99978 | lista: 0 | thread: 4
Numero: 99979 | lista: 1 | thread: 4
Numero: 99980 | lista: 1 | thread: 4
Numero: 99981 | lista: 0 | thread: 8
Numero: 99982 | lista: 0 | thread: 8
Numero: 99983 | lista: 1 | thread: 5
Numero: 99984 | lista: 0 | thread: 4
Numero: 99985 | lista: 0 | thread: 4
Numero: 99986 | lista: 0 | thread: 4
Numero: 99987 | lista: 0 | thread: 4
Numero: 99988 | lista: 1 | thread: 4
Numero: 99989 | lista: 1 | thread: 4
Numero: 99990 | lista: 1 | thread: 4
Numero: 99991 | lista: 0 | thread: 4
Numero: 99992 | lista: 0 | thread: 4
Numero: 99993 | lista: 0 | thread: 4
Numero: 99994 | lista: 0 | thread: 4
Numero: 99995 | lista: 0 | thread: 4
Numero: 99996 | lista: 1 | thread: 8
Numero: 99997 | lista: 1 | thread: 4
Numero: 99998 | lista: 0 | thread: 8
Numero: 99999 | lista: 1 | thread: 4
Numero: 100000 | lista: 0 | thread: 4
Lista 1: 49943 elementos
Lista 2: 50057 elementos
Total inserido: 100000 (esperado: 100000)
./main_2L 100000 113,84s user 0,63s system 1139% cpu 10,049 total
on main !2
```

Figura 3: main_two_list.c com N = 100000

Alterado o código para a criação de M listas e N inserções, para que fosse possível uma entrada do usuário para definir a quantidade de listas a serem criadas, e testar o caso onde não será possível usar regiões críticas nomeadas e será necessário utilizar as funções do lock.

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_9 _ □ X
> export OMP_NUM_THREADS=12; time ./main_NL 4 4
Quantidade de numeros: 4 | Numero de Listas: 4
Numero: 2 | lista: 0 | thread: 3
Numero: 1 | lista: 3 | thread: 11
Numero: 3 | lista: 3 | thread: 6
Numero: 4 | lista: 2 | thread: 5
Lista 0: 1 elementos
Lista 1: 0 elementos
Lista 2: 1 elementos
Lista 3: 2 elementos
Total inserido: 4 (esperado: 4)
./main_NL 4 4 0,07s user 0,00s system 699% cpu 0,010 total
```

Figura 4: main_n_lists.c com N = 4 e M = 4

Foi feito um segundo teste utilizando a implementação atual criando apenas 2 listas com 20 elementos:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_9 _ □ X
> export OMP_NUM_THREADS=12; time ./main_NL 20 2
Quantidade de numeros: 20 | Numero de Listas: 2
Numero: 3 | lista: 1 | thread: 4
Numero: 9 | lista: 1 | thread: 6
Numero: 4 | lista: 1 | thread: 11
Numero: 1 | lista: 0 | thread: 9
Numero: 12 | lista: 1 | thread: 4
Numero: 13 | lista: 1 | thread: 3
Numero: 5 | lista: 1 | thread: 2
Numero: 6 | lista: 0 | thread: 8
Numero: 14 | lista: 1 | thread: 9
Numero: 7 | lista: 0 | thread: 5
Numero: 11 | lista: 1 | thread: 7
Numero: 2 | lista: 1 | thread: 1
Numero: 17 | lista: 1 | thread: 6
Numero: 16 | lista: 0 | thread: 4
Numero: 18 | lista: 1 | thread: 8
Numero: 19 | lista: 1 | thread: 5
Numero: 8 | lista: 1 | thread: 10
Numero: 10 | lista: 1 | thread: 0
Numero: 15 | lista: 1 | thread: 11
Numero: 20 | lista: 1 | thread: 4
Lista 0: 4 elementos
Lista 1: 16 elementos
Total inserido: 20 (esperado: 20)
./main_NL 20 2 0,12s user 0,00s system 766% cpu 0,016 total
```

Figura 5: main_n_lists.c com N = 20 e M = 2

Agora utilizando os mesmo 20 elementos porém criando 10 listas:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_9 _ □ X
> export OMP_NUM_THREADS=12; time ./main_NL 20 10
Quantidade de numeros: 20 | Numero de Listas: 10
Numero: 6 | lista: 0 | thread: 11
Numero: 9 | lista: 7 | thread: 4
Numero: 10 | lista: 9 | thread: 11
Numero: 11 | lista: 0 | thread: 11
Numero: 12 | lista: 0 | thread: 11
Numero: 13 | lista: 0 | thread: 11
Numero: 14 | lista: 6 | thread: 11
Numero: 15 | lista: 1 | thread: 11
Numero: 16 | lista: 1 | thread: 11
Numero: 18 | lista: 5 | thread: 11
Numero: 19 | lista: 4 | thread: 11
Numero: 20 | lista: 6 | thread: 11
Numero: 5 | lista: 0 | thread: 5
Numero: 1 | lista: 0 | thread: 6
Numero: 2 | lista: 3 | thread: 1
Numero: 7 | lista: 2 | thread: 9
Numero: 8 | lista: 4 | thread: 2
Numero: 17 | lista: 4 | thread: 4
Numero: 3 | lista: 1 | thread: 10
Numero: 4 | lista: 6 | thread: 7
Lista 0: 6 elementos
Lista 1: 3 elementos
Lista 2: 1 elementos
Lista 3: 1 elementos
Lista 4: 3 elementos
Lista 5: 1 elementos
Lista 6: 3 elementos
Lista 7: 1 elementos
Lista 8: 0 elementos
Lista 9: 1 elementos
Total inserido: 20 (esperado: 20)
./main_NL 20 10 0,15s user 0,00s system 697% cpu 0,022 total
```

Figura 6: main_n_lists.c com N = 20 e M = 10

Para fim de testes, foi feita uma execução com N = 100000 e M = 2, para testar a deterioração do código à medida que essas entradas aumentam e comparar com a primeira implementação usando o critical nomeado. Por fim com N = 100000 e M = 12.

```

felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_9
Numero: 99963 | lista: 1 | thread: 7
Numero: 99964 | lista: 1 | thread: 8
Numero: 99965 | lista: 0 | thread: 8
Numero: 99966 | lista: 1 | thread: 0
Numero: 99967 | lista: 1 | thread: 1
Numero: 99968 | lista: 0 | thread: 0
Numero: 99969 | lista: 1 | thread: 8
Numero: 99970 | lista: 1 | thread: 8
Numero: 99971 | lista: 1 | thread: 11
Numero: 99972 | lista: 1 | thread: 1
Numero: 99973 | lista: 1 | thread: 0
Numero: 99974 | lista: 0 | thread: 1
Numero: 99975 | lista: 1 | thread: 1
Numero: 99976 | lista: 1 | thread: 11
Numero: 99977 | lista: 0 | thread: 11
Numero: 99978 | lista: 1 | thread: 1
Numero: 99979 | lista: 1 | thread: 1
Numero: 99980 | lista: 0 | thread: 0
Numero: 99981 | lista: 1 | thread: 0
Numero: 99982 | lista: 1 | thread: 0
Numero: 99983 | lista: 0 | thread: 8
Numero: 99984 | lista: 1 | thread: 11
Numero: 99985 | lista: 1 | thread: 8
Numero: 99986 | lista: 0 | thread: 1
Numero: 99987 | lista: 1 | thread: 0
Numero: 99988 | lista: 1 | thread: 1
Numero: 99989 | lista: 1 | thread: 11
Numero: 99990 | lista: 1 | thread: 8
Numero: 99991 | lista: 1 | thread: 11
Numero: 99992 | lista: 1 | thread: 11
Numero: 99993 | lista: 1 | thread: 1
Numero: 99994 | lista: 0 | thread: 1
Numero: 99995 | lista: 0 | thread: 11
Numero: 99996 | lista: 0 | thread: 11
Numero: 99997 | lista: 0 | thread: 1
Numero: 99998 | lista: 1 | thread: 11
Numero: 99999 | lista: 0 | thread: 1
Numero: 100000 | lista: 1 | thread: 0
Lista 1: 49983 elementos
Lista 2: 50017 elementos
Total inserido: 100000 (esperado: 100000)
./main_2L 100000 2 112,73s user 0,61s system 1135% cpu 9,978 total

```

Figura 7: main_n_lists.c com N = 100000 e M = 2

Por fim com N = 100000 e M = 30.

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_9 _ □ X
Numero: 99995 | lista: 4 | thread: 4
Numero: 99988 | lista: 4 | thread: 11
Numero: 99996 | lista: 11 | thread: 6
Numero: 99997 | lista: 21 | thread: 1
Numero: 99998 | lista: 8 | thread: 5
Numero: 99999 | lista: 29 | thread: 10
Numero: 99985 | lista: 0 | thread: 0
Numero: 99994 | lista: 29 | thread: 8
Numero: 100000 | lista: 12 | thread: 3
Numero: 99991 | lista: 22 | thread: 9
Lista 0: 3326 elementos
Lista 1: 3363 elementos
Lista 2: 3382 elementos
Lista 3: 3397 elementos
Lista 4: 3274 elementos
Lista 5: 3261 elementos
Lista 6: 3328 elementos
Lista 7: 3333 elementos
Lista 8: 3330 elementos
Lista 9: 3442 elementos
Lista 10: 3358 elementos
Lista 11: 3381 elementos
Lista 12: 3240 elementos
Lista 13: 3265 elementos
Lista 14: 3299 elementos
Lista 15: 3378 elementos
Lista 16: 3248 elementos
Lista 17: 3351 elementos
Lista 18: 3268 elementos
Lista 19: 3397 elementos
Lista 20: 3493 elementos
Lista 21: 3309 elementos
Lista 22: 3263 elementos
Lista 23: 3438 elementos
Lista 24: 3275 elementos
Lista 25: 3278 elementos
Lista 26: 3357 elementos
Lista 27: 3329 elementos
Lista 28: 3346 elementos
Lista 29: 3291 elementos
Total inserido: 100000 (esperado: 100000)
./main_NL 100000 30 4,62s user 1,09s system 533% cpu 1,070 total
```

Figura 8: main_n_lists.c com N = 100000 e M = 12

5 - Conclusão

A comparação entre as duas abordagens de sincronização evidenciou uma diferença importante: regiões nomeadas utilizando (`#pragma omp critical <nome>`) são resolvidas em tempo de compilação, já que o nome funciona como um identificador, isso faz com que não possa ter mais de uma thread dentro de uma região nomeada, porém podem haver duas threads usando duas regiões nomeadas diferentes. Porém isso traz um problema, que pode ser visto nessa tarefa que é quando são necessárias em M zonas críticas essas que são definidas em tempo de execução, essa abordagem cai por terra pois o crítico nomeado só funciona com dados previamente definidos. Já utilizando o lock se resolve esse problema por serem variáveis comuns e podendo ser alocados em um array cujo tamanho pode ser definido em tempo de execução de forma que qualquer M usado possa ser criado.

Um teste controlado, fixando $M = 2$ nas duas implementações, mostrou que o tempo de execução com região crítica nomeada (10,049s) e com locks explícitos (9,978s) foi praticamente equivalente para $N = 100.000$. Isso indica que a escolha entre critical nomeado e locks não tem, por si só, impacto relevante no desempenho quando o número de recursos protegidos é o mesmo; a vantagem dos locks é de flexibilidade (funcionar com M definido em tempo de execução), não de velocidade.